



Technical Assignment

Job Title: Product Engineering Manager

Business unit/Department: GIT

About AWR Group

At AWR Group, we are at work for a planet that prospers. A world that serves all generations those of today, and those yet to come. We're a group of companies, transforming businesses of today and building purposeful ventures for the future. Our greatest endeavour is to enhance the lives of all generations we touch. We are AWR Group. We embrace generation next.

Assignment Overview

Submission Deadline: 7 days from receipt of this assignment

Format: Code repository + Presentation (PPT/PDF) + Live walkthrough session

Scenario

AW Rostamani (AWR) works with third-party logistics vendors who handle vehicle pick-up and drop-off for customers. Once a trip is initiated, AWR operations teams need **real-time visibility** into vehicle location while in transit — accessible via a web dashboard.

You are required to design, architect, and partially implement this **Vehicle Live Tracking Web System**.

Assumptions You May Work With

1. Vendor drivers use a dedicated interface to publish GPS coordinates while a trip is active.
2. AWR operations staff monitor trips via a **Next.js web dashboard**.
3. Since live GPS may not be available during the assignment, **location data must be simulated** — a trip simulator that moves a marker along a route is acceptable.
4. **Authentication & Authorization** do not need to be implemented in code, but must be covered in the presentation (proposed design is sufficient).
5. You may use any mapping provider (Google Maps, Mapbox, or OpenStreetMap/Leaflet).



Deliverable 1 — Working Code

Next.js Backend & Web Dashboard

Build a Next.js application covering the following:

Backend (API Routes / App Router)

Requirement	Detail
Trip management API	POST /api/trips to create a trip, GET /api/trips/:id to fetch status
Location ingestion endpoint	POST /api/trips/:id/location — receives { lat, lng, timestamp, speed? } from the vendor side
WebSocket or SSE endpoint	Push live location updates to connected web clients without polling
Trip simulator	A server-side utility or API route that emits simulated GPS coordinates at a configurable interval along a predefined route
Data layer	Use an appropriate store (in-memory, Redis, or a lightweight DB) — justify your choice

Frontend (Next.js Web Dashboard)

Requirement	Detail
Active trips list	Show all trips currently in transit with status indicators
Live map view	Display a moving vehicle marker updating in real-time on a map
Trip timeline	Show a log of location pings with timestamps
Vendor simulator panel	A UI control to start/stop a simulated trip and configure update interval
Responsive layout	Must work on both desktop and tablet viewports

Deliverable 2 — Architecture Presentation (PPT/PDF)

Your presentation must cover the following sections. **Depth of thinking here is weighted equally with code quality.**

1. Use Cases & User Journeys

- Vendor driver flow (initiate trip → stream location → end trip)
- AWR operations team flow (monitor dashboard → view trip detail → alerts)

- Admin flow (create/assign trips, view history)

2. System Architecture

- High-level architecture diagram (client → API → real-time layer → data store)
- Justification for real-time mechanism chosen (WebSocket vs SSE vs polling)
- How location data flows from the vendor interface to the AWR dashboard
- How you would handle **scale** (100+ concurrent trips)

3. Integration & Middleware Proposal

- API contract design (request/response schema, versioning strategy)
- Webhook or event-driven approach for notifying downstream systems (e.g., customer SMS on trip start/end)
- Middleware layer considerations (rate limiting, request validation, logging)
- How the system could integrate with AWR's existing operational platforms

4. Architecture Improvement Proposals

Beyond the MVP, propose improvements such as:

- Moving to an event-driven architecture (e.g., message queues for location events)
- Geofencing — trigger alerts when a vehicle enters or exits a defined zone
- Historical trip replay and route deviation detection
- Multi-region or edge deployment for lower latency

5. Performance Optimizations

Address at least **three** of the following:

- Map rendering performance with frequent marker updates
- Batching vs streaming location payloads
- CDN and asset caching strategy for the web dashboard
- Database indexing strategy for location records
- Connection management for WebSocket at scale
- Server-side rendering vs client-side rendering trade-offs in Next.js

6. Security

Cover your proposed approach for:

- Authentication (who are you?) — JWT, OAuth2, or session-based strategy
- Authorization (what can you do?) — role-based access: Vendor vs AWR Ops vs Admin
- Data in transit — TLS, secure WebSocket (WSS)

- Data at rest — encryption for location records
- API abuse prevention — rate limiting, input validation, anomaly detection

7. Analytics

Propose an analytics strategy covering:

- **Operational metrics:** trip duration, average speed, delay frequency, route efficiency
- **System health:** API response times, WebSocket connection stability, error rates
- **Business metrics:** SLA compliance per vendor, on-time pick-up/drop-off rate
- Tools you would use (Google Analytics, Mixpanel, custom BI dashboard, etc.)

8. Testing Strategy

Layer	Approach
Unit tests	Core business logic — trip state machine, location validation, coordinate calculations
Integration tests	API route testing, WebSocket/SSE connection lifecycle
Component tests	Next.js UI components with React Testing Library
E2E tests	Playwright — happy path trip simulation, real-time map update verification
Performance tests	Load test the location ingestion endpoint (k6 or Artillery)

9. Deployment Strategy

Propose a production deployment plan:

- Hosting platform for Next.js (Vercel, AWS, GCP — justify your choice)
 - CI/CD pipeline design (branch strategy, automated tests, staged rollout)
 - Environment management (dev / staging / production)
 - Monitoring & alerting (uptime, error rates, latency thresholds, WebSocket drop rate)
 - Infrastructure as Code considerations (Terraform, Pulumi, or equivalent)
-

Assessment Criteria



Dimension	Weight	What We Look For
Proposed Solution Quality	35%	Completeness, realism, scalability thinking, trade-off awareness
Next.js Code Quality	35%	Code structure, API design, real-time implementation, error handling
Presentation & Communication	30%	Clarity of diagrams, depth of proposals, ability to defend decisions in Q&A

Tips for Candidates

- A **well-structured README** and clean folder organisation signals engineering management mindset — you are being evaluated on how you would set standards for a team, not just whether your code runs.
- The **presentation carries significant weight** — a candidate who proposes a thoughtful architecture with partial code implementation will score higher than one who ships complete code but cannot articulate design decisions.
- Treat the simulator as a first-class feature, not an afterthought. It demonstrates product thinking.
- Comment explicitly on decisions you would make differently **with more time** — this shows self-awareness and prioritisation skill.